

Tracing in Go

Two separate things both called “tracing.” First: `runtime/trace` for in-process execution traces (goroutine scheduling, GC, syscalls). Second: distributed tracing (OpenTelemetry, spans, trace propagation across services). Both matter; they solve different problems.

Two kinds of trace

	<code>runtime/trace</code>	Distributed tracing (OTel)
Captures	Goroutine scheduling, GC, syscalls, user regions	Cross-service request flow
Granularity	Microseconds, per-event	Per-operation (HTTP call, DB query)
Where viewed	<code>go tool trace</code> (local)	Jaeger, Tempo, Datadog, etc.
Cost	Significant; turn on for diagnosis	Low (sampled); usually always on in prod
Use case	“Why is my Go program slow?”	“Where in this 12-service request flow is the latency?”

Most production work is the second kind. The first is a debugging hammer when you’re stuck.

`runtime/trace`

The standard library’s execution tracer. Captures every scheduling decision the runtime makes.

Capturing a trace

```
import "runtime/trace"

f, _ := os.Create("trace.out")
defer f.Close()

trace.Start(f)
defer trace.Stop()

// ... run your program ...
```

Or via `go test`:

```
go test -trace=trace.out ./...
```

Or from a running process via `net/http/pprof` :

```
import _ "net/http/pprof"

go http.ListenAndServe("localhost:6060", nil)
```

Then:

```
curl -o trace.out 'http://localhost:6060/debug/pprof/trace?seconds=5'
```

Viewing it

```
go tool trace trace.out
```

Opens a local web UI. Tabs include: - **View trace**: every goroutine on a timeline. - **Goroutine analysis**: per-goroutine summary. - **Network blocking profile**: time spent on network I/O. - **Synchronization blocking profile**: time on locks/channels. - **Syscall blocking profile**: time in syscalls. - **Scheduler latency profile**: how long goroutines waited to run. - **User-defined regions**: your `trace.WithRegion` blocks.

User-defined regions and tasks

Annotate sections of code so they show up in the trace:

```
import "runtime/trace"

ctx, task := trace.NewTask(ctx, "http.handler")
defer task.End()

trace.WithRegion(ctx, "db.query", func() {
    rows, _ := db.QueryContext(ctx, q)
    rows.Close()
})

trace.Log(ctx, "user_id", "42")
```

`Task` is a long-running activity (typically a request). `Region` is a synchronous block within a task. Both show up as overlays on the timeline.

When to use runtime/trace

- Goroutine leak suspicion.

- Latency mystery where pprof CPU profile doesn't explain it (often blocked-on-something, which CPU profile doesn't show).
- GC pause investigation.
- "Why is one specific request slow when CPU usage is low?"

It's invasive (overhead is significant), so don't leave it on. Run for seconds at a time.

Distributed tracing concepts

A **trace** is the end-to-end path of a request through your system. It's made of **spans**, each representing one operation (a function call, an outbound HTTP request, a DB query, etc.).

Spans form a tree:

```
trace_id: abc123
├─ span: "GET /api/checkout"      [200 ms]
│   └─ span: "validate request"   [5 ms]
│       └─ span: "db: SELECT cart" [20 ms]
│           └─ span: "rpc: payment.charge" [150 ms]
│               └─ span: "external: stripe" [140 ms]
│                   └─ span: "db: UPDATE order" [15 ms]
```

Each span has: - A unique `span_id`. - A `trace_id` shared with every span in the same trace. - A `parent_span_id` (except the root). - A name. - Start and end timestamps. - Attributes (key-value tags). - Events (timestamped logs within the span). - Status (ok / error).

Propagation

Trace context flows across service boundaries via HTTP headers (or gRPC metadata). The W3C standard:

```
traceparent: 00-abc123def456789...-0123456789abcdef-01
tracestate: vendor=value
```

Service A creates the root span, sends an HTTP request with `traceparent`. Service B reads it, creates a child span with the same `trace_id`, sends its own request with an updated `traceparent`. The chain continues.

This is propagation. Without it, traces stop at every service boundary.

OpenTelemetry in Go

OpenTelemetry (OTel) is the vendor-neutral standard for tracing, metrics, and logs. Replaces OpenTracing and OpenCensus.

Setup

```
import (
    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/sdk/trace"
    "go.opentelemetry.io/otel/exporters/otlp/otlptrace/otlptracegrpc"
    "go.opentelemetry.io/otel/sdk/resource"
    semconv "go.opentelemetry.io/otel/semconv/v1.24.0"
)

func initTracer(ctx context.Context) (func(), error) {
    exp, err := otlptracegrpc.New(ctx)
    if err != nil { return nil, err }

    res, _ := resource.Merge(resource.Default(),
        resource.NewWithAttributes(semconv.SchemaURL,
            semconv.ServiceName("my-service"),
            semconv.ServiceVersion("1.0.0"),
        ),
    )

    tp := trace.NewTracerProvider(
        trace.WithBatcher(exp),
        trace.WithResource(res),
        trace.WithSampler(trace.ParentBased(trace.TraceIDRatioBased(0.1))),
    )
    otel.SetTracerProvider(tp)

    return func() { _ = tp.Shutdown(ctx) }, nil
}
```

What this does: - Creates an OTLP exporter (sends to a collector at `OTEL_EXPORTER_OTLP_ENDPOINT`). - Wraps it in a `TracerProvider` that batches before sending. - Sets a sampler: keep 10% of new traces; honor the parent decision. - Registers the provider globally.

Creating spans

```
tracer := otel.Tracer("my-component")

ctx, span := tracer.Start(ctx, "doSomething")
defer span.End()

span.SetAttributes(
    attribute.String("user.id", userID),
    attribute.Int("items.count", n),
)

if err != nil {
    span.RecordError(err)
    span.SetStatus(codes.Error, "operation failed")
}
```

Three things to remember: 1. Pass `ctx` so child spans can find their parent. 2. `defer span.End()` even on error paths. 3. `SetStatus` and `RecordError` are different; you usually want both on failure.

Span events

A point-in-time log within a span:

```
span.AddEvent("cache miss")
span.AddEvent("retry", trace.WithAttributes(attribute.Int("attempt", 2)))
```

Events show up as markers on the span in the UI. Useful for “what happened during this operation” without making a child span.

Instrumenting HTTP

Outgoing requests

```
import "go.opentelemetry.io/contrib/instrumentation/net/http/otelhttp"

client := http.Client{
    Transport: otelhttp.NewTransport(http.DefaultTransport),
}

req, _ := http.NewRequestWithContext(ctx, "GET", url, nil)
resp, err := client.Do(req)
```

The transport wraps every outgoing request: creates a span, injects `traceparent`, records status.

Incoming requests

```
handler := otelhttp.NewHandler(myHandler, "my-service")
http.ListenAndServe(":8080", handler)
```

The middleware reads `traceparent` from incoming headers, starts a span, attaches it to the request context. Downstream handlers can `tracer.Start(r.Context(), ...)` to create children.

For specific frameworks: - `gin-gonic/contrib/otelgin` - `gorilla/mux + otelmux` - `chi + otelchi`

Instrumenting gRPC

```
import "go.opentelemetry.io/contrib/instrumentation/google.golang.org/grpc/otelgrpc"

// server
grpcServer := grpc.NewServer(
    grpc.StatsHandler(otelgrpc.NewServerHandler()),
)

// client
conn, _ := grpc.NewClient(addr,
    grpc.WithStatsHandler(otelgrpc.NewClientHandler()),
)
```

Same idea: handler reads/writes context, creates spans automatically.

Instrumenting databases

database/sql

```
import "go.nhat.io/otelsql"

driverName, err := otelsql.Register("postgres",
    otelsql.WithAttributes(semconv.DBSystemPostgreSQL),
)
db, err := sql.Open(driverName, dsn)
```

Wraps queries, recording duration and statement (with optional sanitization).

Common ORMs

- `gorm.io/plugin/opentelemetry`
- `bun` 's built-in instrumentation
- `sqlx` works with `otelsql` since it wraps `database/sql`

For Redis (`go-redis`), MongoDB, Kafka, etc., each driver has an otel contrib package.

Sampling

Tracing every request is expensive at scale. Sampling decides which traces to keep.

Sampler types

Sampler	Behavior
AlwaysSample	Keep everything (dev / low volume)
NeverSample	Drop everything
TraceIDRatioBased(p)	Hash-based: keep p fraction (e.g., 0.1 = 10%)
ParentBased(base)	Honor parent's decision; if no parent, use base

Standard production config:

```
trace.WithSampler(trace.ParentBased(trace.TraceIDRatioBased(0.01)))
```

Keep 1% of root traces, then honor whatever upstream decided for child services. This keeps an entire trace either sampled or not; you don't get a service in the middle that dropped half of its spans.

Tail-based sampling

Some backends (Jaeger, Tempo, Datadog) support tail-based sampling: collect all spans, decide later whether to keep based on the final shape (errored? slow? hit a rare path?). Configured at the collector, not in your app.

Correlation: logs + traces + metrics

The three pillars only work if they cross-reference.

Logs reference trace ID

```
span := trace.SpanFromContext(ctx)
sc := span.SpanContext()
logger := slog.Default().With(
    "trace_id", sc.TraceID().String(),
    "span_id",  sc.SpanID().String(),
)
logger.Info("processing")
```

In the aggregator: click a log line, jump to the trace. Click a slow span, see its logs.

Metrics with exemplars

Some metric backends (Prometheus + Tempo, Datadog) support exemplars: a metric data point can carry a sample trace ID. When you see a latency spike on a histogram, click to a specific trace that

contributed to that spike.

```
counter.Add(ctx, 1) // OTel metric; trace ID picked up from ctx automatically
```

Custom propagation

OTel uses W3C trace context by default. To support B3 (Zipkin) or other formats:

```
import "go.opentelemetry.io/contrib/propagators/b3"

otel.SetTextMapPropagator(b3.New())
```

Or combine multiple:

```
otel.SetTextMapPropagator(propagation.NewCompositeTextMapPropagator(
    propagation.TraceContext{},
    propagation.Baggage{},
    b3.New(),
))
```

Baggage is OTel's way to carry arbitrary key/value pairs alongside the trace context across services. Use sparingly; every key is in every request.

What to put on spans

Always

- `service.name`, `service.version` (set on resource, not span).
- HTTP: method, path template (not full URL with IDs!), status code, response size.
- DB: system (postgres, mysql), statement (sanitized), table, rows affected.
- RPC: service name, method.
- Errors: as both `RecordError` and `SetStatus(Error, "...")`.

Usually

- User ID, tenant ID, account ID (PII-aware; check policy).
- Cache hit/miss.
- Retry attempt number.

Never

- Full credit card numbers, passwords, tokens.
- Large blobs (request body, response body). Use events for size only.
- Secrets in error messages.

Use semantic conventions (`semconv.HTTPStatusCode(...)`) so the backend understands the data.

Performance cost of tracing

For a typical service: - Span creation: 100-500 ns. - Attribute setting: 50-100 ns per attribute. - Export: batched and async; negligible to the request path. - Sampler decision: 50-100 ns at the root span.

With 10-20 spans per request and 1-10% sampling, tracing adds maybe 1-5 microseconds to a hot path. Usually fine; benchmark if you doubt.

The big cost is the network: shipping spans to the backend. Use a local OTel collector that batches before forwarding, so your app sends UDP/gRPC to localhost and the collector handles the wide-area send.

Common architecture

```
[app] -> [otel-collector sidecar] -> [otel-collector gateway] -> [Tempo/Jaeger]
```

The sidecar: - Receives OTLP from the app (localhost, no auth needed). - Buffers during backend outages. - Adds resource attributes (host, pod, region). - Drops invalid data.

The gateway: - Aggregates from many sidecars. - Applies tail sampling. - Forwards to storage.

The app code stays simple: send to localhost via OTLP and forget.

pprof, runtime/trace, OTel: which when

Symptom	Tool
“What’s using all the CPU?”	<code>pprof -cpu</code>
“Where’s the memory going?”	<code>pprof -heap</code>
“Why is this goroutine stuck?”	<code>pprof -goroutine</code> , or <code>runtime/trace</code>
“Why are GC pauses long?”	<code>runtime/trace</code> (or <code>GODEBUG=gctrace=1</code>)
“Where does this cross-service request go?”	OTel distributed tracing
“Which DB call is slow in production?”	OTel spans on the DB driver
“Why is one endpoint occasionally slow?”	OTel + tail sampling for slow traces
“What does the scheduler do under load?”	<code>runtime/trace</code>

Best practices

1. **Set service name and version on the resource.** Everything downstream filters by these.
2. **Use semantic conventions.** Don’t invent your own attribute names.
3. **Propagate context everywhere.** A function that does I/O takes `ctx context.Context` .
4. **Defer `span.End()` right after `tracer.Start()` .** Easy to forget on error paths.
5. **Sample at the root.** `ParentBased(TraceIDRatioBased(p))` keeps traces consistent.
6. **Send to a local collector, not directly to the backend.** Decouples app from network.
7. **Add trace IDs to logs.** Cross-tool correlation is the win.
8. **Don’t tag PII** unless you have a policy and your backend supports redaction.
9. **One tracer per package** (`otel.Tracer("github.com/me/pkg")`). The name shows up in queries.
10. **Wrap every external call in a span.** That’s where minutes hide.

Common gotchas

Gotcha	Symptom	Fix
Forgot <code>span.End()</code>	Spans never finalized; lost or hung	<code>defer span.End()</code> first thing
Passed wrong context	Span has no parent (orphan trace)	Always thread <code>ctx</code> through
Trace context not propagated	Traces stop at service boundary	Use <code>otelhttp</code> / <code>otelgrpc</code> middleware
Sampling at child but not parent	Half-sampled traces in UI	<code>ParentBased</code> sampler
Spans with high-cardinality names	Tracing backend explodes	Use path templates, not full paths
Logging sensitive data as span attribute	Compliance violation	Redact before adding
Tracing disabled, <code>tracer.Start</code> is noop	Spans missing in dev	Check tracer provider is registered
Outgoing HTTP without <code>otelhttp</code> transport	Downstream services start fresh traces	Wrap <code>http.Transport</code>
Trying to read traces in code	API isn't for that	Tracing is fire-and-forget; query the backend
Massive batch in slow backend	App memory pressure	Tune <code>BatchTimeout</code> and <code>MaxQueueSize</code>
Full request body in span attributes	Backend chokes; cost spike	Log size only
Mixing OpenTracing and OpenTelemetry	Half-instrumented code	Migrate fully to OTel

Quick reference

What	Tool / Code
In-process execution trace	<code>runtime/trace.Start(f)</code>
View execution trace	<code>go tool trace trace.out</code>
Capture via pprof endpoint	<code>/debug/pprof/trace?seconds=5</code>
User region	<code>trace.WithRegion(ctx, "name", func(){...})</code>
User task	<code>trace.NewTask(ctx, "name")</code>
OTel tracer	<code>otel.Tracer("pkg")</code>
Start span	<code>ctx, span := tracer.Start(ctx, "name")</code>
End span	<code>defer span.End()</code>
Set attr	<code>span.SetAttributes(attribute.String("k", v))</code>
Record error	<code>span.RecordError(err); span.SetStatus(codes.Error, "...")</code>
Add event	<code>span.AddEvent("name")</code>
Span from ctx	<code>trace.SpanFromContext(ctx)</code>
HTTP server middleware	<code>otelhttp.NewHandler(h, "name")</code>
HTTP client transport	<code>otelhttp.NewTransport(rt)</code>
gRPC server handler	<code>grpc.StatsHandler(otelgrpc.NewServerHandler())</code>
Sampler (ratio)	<code>trace.TraceIDRatioBased(0.1)</code>
Sampler (honor parent)	<code>trace.ParentBased(base)</code>
Propagator default	W3C <code>traceparent</code>
Backend exporter	OTLP gRPC: <code>otlptracegrpc.New(ctx)</code>
Trace ID for logs	<code>span.SpanContext().TraceID().String()</code>
Collector pattern	App -> local OTel collector -> gateway -> backend